



FileFlow Studio

Manual de Usuario y Guía de Referencia Completa

FileFlow Studio — Motor Visual de Automatización DAG en .NET 9



Manual de Usuario y Guía de Referencia de Nodos

FileFlow Studio v2.0

Plataforma de Automatización y Procesamiento de Archivos basada en Nodos en .NET 9 y C# 13



Tabla de Contenidos

1. Introducción y Filosofía de Diseño
2. Conceptos Fundamentales del Editor
 - Lienzo de Nodos (Nodify)
 - El Contexto del Archivo (`FileItemContext`)
 - Sub-flujos y Macros Multinivel (Breadcrumbs)
 - Badges de Telemetría en Tiempo Real
3. Modos de Ejecución y Seguridad de Datos
 - Ejecución Normal
 - Modo Simulación Virtual ("Dry Run")
 - Sistema de Rollback y Papelera de Windows
 - Depuración Interactiva con Breakpoints
4. Motor de Tokens y Variables Dinámicas
 - Sintaxis de Tokens
 - Proveedores y Funciones Nativas
5. Catálogo Exhaustivo de Nodos
 - Categoría 1: FileSystem (E/S de Disco)
 - Categoría 2: Logic (Control de Flujo)

- Categoría 3: Hashing (Integridad y Deduplicación)
- Categoría 4: Archives (Compresión y Desempaquetado)
- Categoría 5: Images & Media (Multimedia y EXIF)
- Categoría 6: Integrations (CLI y Webhooks)

6. Flujos de Ejemplo y Plantillas Predefinidas

1. Introducción y Filosofía de Diseño

FileFlow Studio es una plataforma de automatización de archivos diseñada para crear pipelines visuales de procesamiento masivo, inspirada en herramientas como *n8n*, *ComfyUI* y *Node-RED*.

Características Principales:

- **Modularidad por Plugins:** Cada grupo de funcionalidades vive en un plugin desacoplado (`FileFlow.Plugin.*`).
 - **Seguridad Garantizada:** Simulación previa mediante *Dry Run* y borrado recuperable con la Papelera de reciclaje de Windows.
 - **Rendimiento Asíncrono en .NET 9:** Canales reactivos sin bloqueo de interfaz gráfica.
 - **Sub-flujos Anidados:** Capacidad de crear sub-grafos dentro de macros reutilizables con navegación por migas de pan (*Breadcrumbs*).
-

2. Conceptos Fundamentales del Editor

Lienzo de Nodos (Nodify)

El lienzo permite arrastrar nodos desde la **Caja de Herramientas (Toolbox)** izquierda. Cada nodo dispone de:

- **Puertos de Entrada (Input Ports):** Ubicados a la izquierda. Reciben archivos o señales de control.
- **Puertos de Salida (Output Ports):** Ubicados a la derecha. Emiten archivos procesados o desvíos condicionales.
- **LED de Estado:** Indicador luminoso que refleja el estado de ejecución (`Idle`, `Running`, `Completed`, `Paused`, `Faulted`).
- **Punto de Interrupción (Breakpoint):** Clic en el círculo rojo superior para pausar la ejecución en ese nodo.

El Contexto del Archivo (`FileItemContext`)

Un registro inmutable/transmutable que viaja a través de los cables del grafo conteniendo:

- `CurrentPath`: Ruta actual del archivo en el paso del pipeline.
- `OriginalPath`: Ruta de origen con la que inició el flujo.
- `FileSizeBytes`: Tamaño exacto en bytes.

- **Metadata** : Diccionario de clave-valor enriquecido por los nodos (**Exif:** , **Hash:** , **Cli:*** , etc.).
- **ExecutionLog** : Historial de transformaciones aplicadas.

Sub-flujos y Macros Multinivel (Breadcrumbs)

Los sub-flujos permiten encapsular conjuntos de nodos dentro de un único nodo compuesto:

1. Para editar el sub-flujo, haz doble clic en el nodo o pulsa "**Abrir Sub-flujo**".
2. El lienzo mostrará el sub-grafo interno y una barra superior de migas de pan (*Breadcrumbs*): **Root Workflow > Sub-flujo A > Sub-flujo B**.
3. Al hacer clic en un nivel anterior de la barra de migas de pan, los cambios se guardan y vuelves al flujo padre.

Badges de Telemetría en Tiempo Real

Durante la ejecución, cada cable de conexión entre nodos actualiza dinámicamente un contador de elementos procesados (ej. **⚡ 1,250 items**), permitiendo identificar cuellos de botella o desvíos condicionales en vivo.

3. Modos de Ejecución y Seguridad de Datos

Ejecución Normal

Pulsa el botón verde "**▶ Ejecutar Flujo**" en la barra superior. El motor procesará todos los archivos en paralelo o según las restricciones de concurrencia adaptativa configuradas.

Modo Simulación Virtual ("Dry Run")

1. Marca la casilla "**Dry Run**" o pulsa "**Simulación Virtual**".
2. El motor recorrerá todo el grafo resolviendo nombres, rutas, hashes y condiciones **sin modificar, mover ni eliminar ningún archivo real en el disco**.
3. En la consola inferior se generará un desglose de todas las acciones planificadas (**PlannedAction**).

Sistema de Rollback y Papelera de Windows

- **Borrado Seguro:** Todos los nodos de eliminación utilizan la API nativa de Windows Shell (**SHFileOperation**), enviando los ficheros a la Papelera de reciclaje.
- **Deshacer (Rollback):** Si tras ejecutar un flujo deseas revertir los cambios, pulsa el botón "**↶ Deshacer**" en la barra superior. El sistema deshacerá los renombrados y movimientos en orden LIFO (último en ejecutarse, primero en deshacerse).

Depuración Interactiva con Breakpoints

- Pulsa "**🐛 Depurar Flujo**".
- La ejecución se detendrá automáticamente cuando un archivo alcance un nodo con breakpoint activo o cuando ocurra una excepción.

- Usa **"Paso a Paso (F10)"** para avanzar un nodo a la vez e inspecciona el diff de metadatos en el panel lateral **Inspector de Nodos**.

4. Motor de Tokens y Variables Dinámicas

Cualquier campo de texto, carpeta o plantilla de nombre admite variables dinámicas encerradas entre llaves { ... }.

Sintaxis General:

{Dominio:Clave:Modificador} o {Variable}

Tabla de Tokens Disponibles:

Token	Ejemplo de Salida	Descripción
:---	:---	:---
{FileName}	documento.pdf	Nombre del archivo con extensión
{FileNameNoExt}	documento	Nombre sin extensión
{Ext}	pdf	Extensión sin punto
{CurrentDir}	C:\MisArchivos	Directorio actual
{ParentDir}	MisArchivos	Nombre de la carpeta contenedora
{CreationDate:yyyyMMdd}	20260822	Fecha de creación del archivo con formato
{ModifiedDate:yyyy-MM-dd}	2026-08-22	Fecha de última modificación
{Now:yyyyMMdd_HH:mm:ss}	20260822_143000	Fecha y hora actual del sistema
{FileSize:MB}	14.50	Tamaño en Megabytes
{FileSize:KB}	14848.0	Tamaño en Kilobytes
{Hash:SHA256}	e3b0c442 ...	Hash SHA-256 completo
{Hash:SHA256:8}	e3b0c442	Hash SHA-256 truncado a 8 caracteres
{Hash:MD5:6}	d41d8c	Hash MD5 truncado
{Exif:CameraModel}	Nikon Z8	Modelo de cámara desde metadatos EXIF
{Exif:Make}	NIKON	Fabricante de cámara
{Env:USERPROFILE}	C:\Users\ricardo	Variable de entorno del sistema operativo
{Index:D4}	0001 , 0002	Contador secuencial en el lote

5. Catálogo Exhaustivo de Nodos

Categoría 1: FileSystem (E/S de Disco)

1. FolderSourceNode (Origen de Carpeta)

- **Propósito:** Inicia el flujo escaneando un directorio y emitiendo cada archivo encontrado.
- **Puertos:**
- **Outputs:** FileOut (FileItemContext), Completed (Señal de fin).
- **Parámetros:**
- SourceFolder : Ruta de la carpeta a escanear (admite tokens).
- SearchPattern : Filtro de archivos (ej. . , .jpg; .png).
- IncludeSubdirectories : true para escaneo recursivo.

2. AdvancedRenamerNode (Renombrador Avanzado con Pipeline de Métodos y Tokens)

- **Propósito:** Transforma masivamente nombres de archivos y carpetas mediante un pipeline de hasta **9 métodos acumulativos secuenciales**, emulando las capacidades profesionales de *Advanced Renamer*.
- **Puertos:**
- **Inputs:** In (FileItemContext)
- **Outputs:** Out (Renombrado exitoso), Skipped (Omitido por colisión), Error (Fallo en E/S o validación)
- **Parámetros y Métodos del Pipeline:**
- CollisionStrategy : Estrategia atómica de resolución de colisiones:
- AutoIncrement : Añade automáticamente sufijos incrementales (_1 , _2 , _3) sin bloquear el lote concurrente.
- Overwrite : Sobrescribe el destino si ya existe.
- Skip : Omite el archivo y lo desvía por el puerto Skipped .
- Fail : Interrumpe la ejecución con error controlado.
- MethodSteps : Pipeline JSON de métodos acumulativos que se aplican en orden secuencial:

1. **Nuevos Nombres / Plantillas:** Sustitución total o parcial basada en plantillas con etiquetas dinámicas (o {Tag}).

2. **Búsqueda y Reemplazo:** Búsqueda de subcadenas o patrones mediante **Regex** con grupos de captura (\$1 , \$2), distinción de mayúsculas y reemplazo selectivo o global.

3. **Inserción de Texto:** Agrega cadenas en posiciones absolutas o relativas (desde el inicio o desde el final del nombre).

4. **Eliminación de Caracteres:** Suprime rangos de caracteres por conteo o posiciones relativas.

5. **Conversión de Mayúsculas / Minúsculas:** `Lowercase` , `Uppercase` , `TitleCase` (Tipo Título), `SentenceCase` (Tipo Oración) o `CapitalizeFirst` .

6. **Numeración Incremental:** Generador de secuencias numéricas con valor inicial, incremento, relleno de ceros (*padding*, ej. `001`) y condición de reinicio (*DirectoryChange*, *MetadataChange* o *Never*).

7. **Tabla de Sustituciones (Replace List):** Mapeo de pares clave-valor para reemplazos masivos o depuración de palabras clave.

8. **Limpieza, Recorte y Normalización:** Recorte de espacios en extremos (*Trim*), colapso de espacios múltiples, sanitización de caracteres ilegales de Windows (`\` / : * ? " < > |) y normalización Unicode (`NFC` , `NFD` , `NFKC` , `NFKD`).

9. **Normalización y Relleno de Ceros (Padding) en Números:** Estandariza dígitos existentes en secuencias (`1 - pepe.jpg` $\rightarrow$ `01 - pepe.jpg`), temporadas y episodios de series (`serie guapa 1x1.mov` $\rightarrow$ `serie guapa 1x01.mov` , `S1E2` $\rightarrow$ `S01E02` , `Cap. 2` $\rightarrow$ `Cap. 02`), capítulos y pistas de audio.

- **Asistente y Probador Visual de Expresiones Regulares (Regex Studio):** Botón  `Regex ...` disponible en todos los campos que admiten expresiones regulares (Búsqueda/Reemplazo, Eliminación, Normalización Numérica y Tabla de Sustituciones). Permite:
 - Probar expresiones en tiempo real con nombres de archivo de muestra.
 - Inspeccionar el desglose de grupos de captura (`$1` , `$2` , `${grupo}`).
 - Simular el texto resultante tras el reemplazo.
 - Cargar patrones predefinidos (Series/Episodios, Fechas ISO/Europeas, Timestamps, Limpieza de corchetes/paréntesis, Códecs).
 - Guardar patrones favoritos personalizados con persistencia automática en el perfil de usuario.
- **Estudio Visual Integrado (Studio):** Botón  `Pipeline de Métodos ...` en la tarjeta del nodo y en el inspector para abrir el editor visual interactivo con **Live Preview en tiempo real** y catálogo de **Presets integrados** (Fotografía EXIF, Música ID3, Web/SEO, Documentos Empresariales).

3. `FileRelocatorNode` (Reubicador y Copiador de Archivos)

- **Propósito:** Mueve o copia archivos a rutas calculadas dinámicamente, verificando la integridad binaria mediante hash SHA-256.
- **Puertos:**
 - *Inputs:* `In`
 - *Outputs:* `Out` , `Error`
- **Parámetros:**
 - `Operation` : `Move` (Mover) o `Copy` (Copiar).
 - `DestinationDirectory` : Ruta destino (ej. `{SourceDir}\{Year}\{Month}`).
 - `VerifyIntegrity` : `true` para comprobar que el hash origen y destino coincidan.
 - `CreateDirectories` : `true` para crear carpetas automáticamente si no existen.

4. SafeRecycleDeleteNode (Borrado Seguro a Papelera)

- **Propósito:** Envía archivos a la Papelera de reciclaje de Windows mediante la API nativa del Shell (recuperable y con soporte de Deshacer).
- **Puertos:**
 - *Inputs:* In
 - *Outputs:* Deleted , Error
- **Parámetros:**
 - DeleteOriginalPath : true para borrar el archivo original de entrada, false para la ruta actual.

5. EmptyDirectoryCleanerNode (Limpiador de Carpetas Vacías)

- **Propósito:** Recorre un directorio y elimina de forma recursiva todas las carpetas que hayan quedado vacías tras un procesamiento.
- **Puertos:**
 - *Inputs:* TriggerIn
 - *Outputs:* Out , Error
- **Parámetros:**
 - TargetDirectory : Directorio a limpiar (ej. {SourceDir}).
 - Recursive : true para analizar subcarpetas en profundidad.
 - IgnoreHiddenSystemFiles : true para considerar vacía una carpeta que solo contenga Thumbs.db o .DS_Store .

6. OperationReportNode (Reporte Visual de Operaciones)

- **Propósito:** Genera un informe visual interactivo y estético con la trazabilidad completa del ciclo de vida y transformaciones de cada archivo (desde origen hasta destino con pasos y metadatos), agrupado jerárquicamente por la estructura de carpetas de origen.
- **Puertos:**
 - *Inputs:* In
 - *Outputs:* Out (Reenvía el archivo sin modificar), Report (Emite el archivo de reporte generado), Error
- **Parámetros:**
 - ReportFormat : Desplegable con HTML , Markdown , Text , JSON , CSV (default: HTML).
 - ReportScope : Consolidated (informe único para todo el lote), PerFile (reporte individual adjunto) o Both (ambos).
 - GroupBy : Criterio de agrupación dinámica en el reporte:
 - Directory (Por Defecto): Acordeón interactivo con carpetas colapsables, conteo de archivos, volumen y estados por subdirectorio.
 - Flat : Listado plano secuencial sin agrupadores.
 - Extension : Agrupación por tipo/formato de archivo (.jpg , .pdf , etc.).

- **Status** : Agrupación por archivos exitosos vs con errores/alertas.
- **DestinationFolder** : Ruta o plantilla destino (default: `{RelativeDir}\Output`).
- **ReportFileName** : Plantilla de nombre (default: `Reporte_Ejecucion_{Date:yyyyMMdd_HH:mm:ss}`).
- **Theme** : `ModernDark` (modo oscuro moderno con tarjetas y timeline) o `CleanLight` .
- **AutoOpenReport** : `true` para abrir automáticamente el reporte en el navegador/visor predeterminado al culminar.
- **IncludeMetadata** : `true` para incluir tablas desplegables con todos los atributos EXIF/Hash/Tags.

Categoría 2: Logic (Control de Flujo)

1. **BatchBufferNode** (Agrupador de Lotes)

- **Propósito:** Acumula archivos en memoria hasta alcanzar \$N\$ elementos o un tamaño total en MB antes de emitirlos juntos.
- **Puertos:**
 - *Inputs:* `ItemIn` , `ForceFlush`
 - *Outputs:* `ItemOut` , `BatchCompleted`
- **Parámetros:**
 - **BatchSize** : Número de archivos por lote (ej. `10` , `50`).
 - **MaxBatchSizeBytes** : Tamaño máximo acumulado antes de disparar el lote.

2. **ThrottleDelayNode** (Control de Tasa y Pausa)

- **Propósito:** Regula la velocidad del flujo introduciendo una pausa en milisegundos entre archivos para evitar saturación de I/O.
- **Puertos:**
 - *Inputs:* `In`
 - *Outputs:* `Out`
- **Parámetros:**
 - **DelayMilliseconds** : Tiempo de espera por archivo (ej. `250` ms).

3. **ForkJoinBarrierNode** (Barrera de Sincronización)

- **Propósito:** Bifurca un archivo hacia múltiples ramas paralelas (`Fork1` , `Fork2`) y espera a que todas terminen para emitir `AllCompleted` .
- **Puertos:**
 - *Inputs:* `In` , `Branch1_Done` , `Branch2_Done`
 - *Outputs:* `Fork1` , `Fork2` , `AllCompleted`
- **Parámetros:**
 - **RequiredBranchesCount** : Número de ramas a sincronizar (default: `2`).

4. SwitchCaseNode (Enrutador Condicional Dinámico)

- **Propósito:** Evalúa una expresión o extensión del archivo y lo enruta dinámicamente hacia uno de sus casos configurados o hacia el puerto `Default`.
- **Puertos:**
 - *Inputs:* `In`
 - *Outputs:* Dinámicos según los casos configurados (ej. `Imagenes`, `Videos`, `Documentos`) + `Default`.
- **Parámetros y Casos Dinámicos:**
 - *Expression*: Variable o token a evaluar (ej. `{Ext}`, `{ParentDir}`, `{FileSize:MB}`).
 - **Botón + Caso**: Añade dinámicamente nuevas condiciones al nodo y crea automáticamente el puerto de salida correspondiente.
 - **Nombre del Caso**: Define tanto la etiqueta del caso como el nombre visual del puerto de salida.
 - **Patrón**: Lista de extensiones o textos separados por punto y coma (ej. `jpg;jpeg;png;webp`).
 - **Puerto Default**: Captura automáticamente cualquier archivo que no coincida con ninguno de los casos definidos.

5. ExpressionFilterNode (Filtro por Condición Lógica)

- **Propósito:** Evalúa condiciones numéricas o de texto sobre propiedades (`SizeMB`, `Ext`, `CreationDate`) y desvía por `True` o `False`.
- **Puertos:**
 - *Inputs:* `In`
 - *Outputs:* `True`, `False`
- **Parámetros:**
 - *Property*: Propiedad a evaluar (`SizeMB`, `Ext`, etc.).
 - *Operator*: `>`, `<`, `≥`, `≤`, `==`, `≠`, `Contains`.
 - *ComparisonValue*: Valor de comparación (ej. `50`).

Categoría 3: Hashing (Integridad y Deduplicación)

1. HashCalculatorNode (Calculador de Hash Criptográfico)

- **Propósito:** Calcula el checksum del contenido del archivo y lo inyecta en `Metadata["Hash:"]` y `{Hash: }`.
- **Puertos:**
 - *Inputs:* `In`
 - *Outputs:* `Out`, `Error`
- **Parámetros:**
 - *Algorithm*: `SHA256`, `MD5`, `SHA512`, `SHA1`.
 - *StoreInMetadataKey*: Clave de destino (ej. `Hash:SHA256`).

2. `DeduplicationFilterNode` (Filtro de Deduplicación por Hash)

- **Propósito:** Compara el hash del contenido en el lote actual y separa archivos originales de copias duplicadas.
 - **Puertos:**
 - *Inputs:* `In`
 - *Outputs:* `Unique` (Archivo único), `Duplicate` (Archivo duplicado), `Error`
 - **Parámetros:**
 - `HashMetadataKey` : Clave de hash a verificar (ej. `Hash:SHA256`).
-

Categoría 4: Archivos (Compresión y Desempaquetado)

1. `SmartUnpackNode` (Descompresión Inteligente)

- **Propósito:** Descomprime archivos ZIP, RAR, 7z o TAR extrayendo su contenido a una carpeta destino.
- **Puertos:**
- *Inputs:* `In`
- *Outputs:* `ExtractedFile` , `Done` , `Error`
- **Parámetros:**
- `DestinationPath` : Carpeta donde extraer los archivos.
- `FlattenHierarchy` : `true` para extraer todos los archivos en el mismo nivel.

2. `ArchiveFilterNode` (Filtro de Archivos Comprimidos)

- **Propósito:** Separa archivos comprimidos válidos de otros tipos de archivo.
 - **Puertos:**
 - *Inputs:* `In`
 - *Outputs:* `ArchiveOut` , `NonArchiveOut`
-

Categoría 5: Images & Media (Multimedia y EXIF)

1. `ImageOptimizerNode` (Optimizador y Escalador de Imágenes)

- **Propósito:** Comprime, convierte y redimensiona imágenes a formatos modernos (**WebP, JPEG, PNG**) con cálculo automático de porcentaje de ahorro de espacio (%) y control avanzado de aspecto y escala.
- **Puertos:**
- *Inputs:* `In`
- *Outputs:* `Out` , `Error`
- **Parámetros:**

- **SizeMode** : Modo de cálculo (`"Pixels"` para dimensiones exactas en píxeles o `"Percentage"` para escala relativa en %).
- **Width** : Ancho objetivo en píxeles. Si se configura solo el ancho y `Height = 0`, el alto se calcula automáticamente para mantener la relación de aspecto.
- **Height** : Alto objetivo en píxeles. Si se configura solo el alto y `Width = 0`, el ancho se calcula automáticamente para mantener la relación de aspecto.
- **ScalePercentage** : Porcentaje de escala (ej. `50.0` para reducir al 50%, `150.0` para agrandar al 150%).
- **ScalePercentageY** : Porcentaje vertical en caso de desear relación de aspecto asimétrica cuando **MaintainAspectRatio** está desactivado.
- **MaintainAspectRatio** : Si es `true`, conserva la relación de aspecto original de la imagen; si es `false`, fuerza las dimensiones especificadas.
- **OnlyDownscale** : Si es `true`, **solo reduce imágenes mayores al tamaño objetivo y no agranda imágenes más pequeñas**, evitando pixelado y aumento innecesario de peso. Si es `false`, escala incondicionalmente todas las imágenes.
- **TargetFormat** : Formato de compresión (`"WebP"`, `"JPEG"`, `"PNG"`).
- **Quality** : Calidad de compresión visual (1-100, default: `80`).
- **OutputDirectory** : Carpeta de destino con soporte de variables (ej. `{RelativeDir}\OptimizedImages`).

2. **ExifMetadataNode** (Extracción Estructurada de Metadatos EXIF)

- **Propósito:** Extrae datos EXIF de cámara (fecha de captura, marca, modelo, ISO, apertura, coordenadas GPS) e inyecta metadatos `{Exif:*}` y `{Date:Taken}` en el archivo.
- **Puertos:**
- *Inputs:* `In`
- *Outputs:* `Out`, `Error`

Categoría 6: Integrations (CLI y Webhooks)

1. **CliExecutionNode** (Ejecutor de Comandos y Procesos CLI)

- **Propósito:** Ejecuta herramientas de consola externas (FFmpeg, PowerShell, Python) pasando argumentos dinámicos con tokens.
- **Puertos:**
- *Inputs:* `In`
- *Outputs:* `Success`, `Failed`
- **Parámetros:**
- **ExecutablePath** : Ruta del ejecutable (ej. `ffmpeg.exe` o `cmd.exe`).
- **ArgumentsTemplate** : Argumentos con tokens (ej. `/c echo Procesando {FileName}`).
- **TimeoutSeconds** : Límite de tiempo en segundos (default: `60`).
- **CaptureOutputToMetadata** : Guarda la salida estándar en `Metadata["Cli:StdOut"]` .

2. WebhookNotificationNode (Notificador Webhook HTTP POST)

- **Propósito:** Dispara notificaciones HTTP POST con cuerpo JSON hacia servicios externos (Discord, Slack, n8n, Zapier).
- **Puertos:**
- *Inputs:* In
- *Outputs:* Out, Failed
- **Parámetros:**
- **Url** : URL del endpoint webhook (ej. `https://discord.com/api/webhooks/ ...`).
- **PayloadTemplate** : Cuerpo JSON con tokens (ej. `{"content": "Archivo {FileName} procesado ({FileSize:MB} MB)"}`).
- **TimeoutSeconds** : Timeout de la petición HTTP (default: 15).

Categoría 7: Documents & PDFs (FileFlow.Plugin.Documents)

1. PdfMergeNode (Unión de Archivos PDF)

- **Propósito:** Combina múltiples archivos PDF en un único documento consolidado al concluir el flujo o por lotes con PdfSharp .
- **Puertos:** In $\rightarrow$ Out, MergedOut .

2. PdfSplitNode (División de PDFs por Páginas)

- **Propósito:** Divide un documento PDF en páginas individuales o rangos específicos.
- **Puertos:** In $\rightarrow$ PageOut, Error .

3. PdfTextExtractorNode (Extracción de Texto y Contenido PDF)

- **Propósito:** Extrae texto estructurado desde documentos PDF mediante PdfPig para búsquedas, clasificación o renombrado por contenido.
- **Puertos:** In $\rightarrow$ Out, Error .

4. PdfMetadataNode (Inspección y Modificación de Metadatos PDF)

- **Propósito:** Lee y actualiza metadatos estándar (Título, Autor, Asunto, Palabras Clave) en documentos PDF.
- **Puertos:** In $\rightarrow$ Out .

Categoría 8: Network & Remote Storage (FileFlow.Plugin.Network)

1. FtpUploadNode (Subida a Servidores FTP / FTPS Seguro)

- **Propósito:** Sube archivos a servidores FTP/FTPS con cifrado explícito o implícito TLS/SSL y reanudación automática.

- **Puertos:** In $\rightarrow$ Success , Failed .

2. SftpUploadNode (Transferencia Segura SSH / SFTP)

- **Propósito:** Transfiere archivos hacia servidores Linux / VPS mediante SSH File Transfer Protocol con soporte para contraseña o clave privada (.pem / .ppk).
- **Puertos:** In $\rightarrow$ Success , Failed .

3. SmbCopyNode (Copia a Recursos Compartidos de Red Local / NAS)

- **Propósito:** Copia archivos a rutas de red UNC (\\Servidor\Carpeta) o almacenamiento NAS con autenticación de dominio o usuario local.
- **Puertos:** In $\rightarrow$ Success , Failed .

4. WebDavUploadNode (Nubes Privadas Nextcloud / ownCloud WebDAV)

- **Propósito:** Publica y sincroniza archivos con servidores WebDAV corporativos o nubes personales.
- **Puertos:** In $\rightarrow$ Success , Failed .

5. RemoteDownloadNode (Descarga de Archivos Remotos HTTP / HTTPS / FTP)

- **Propósito:** Descarga archivos remotos desde Internet o servidores locales incorporándolos directamente al pipeline de procesamiento.
- **Puertos:** In $\rightarrow$ Success , Failed .

Categoría 9: Data, Spreadsheets & Databases (FileFlow.Plugin.Data)

1. ExcelReaderNode (Lector de Hojas Excel)

- **Propósito:** Lee hojas de cálculo .xlsx / .xls en streaming de alta velocidad y emite cada fila con sus columnas en item.Metadata .
- **Puertos:** In (opcional) $\rightarrow$ RowOut .

2. CsvReaderNode (Lector de Archivos CSV / TSV)

- **Propósito:** Parser de archivos delimitados con autodetección de delimitador (, , ; , \t , |) y compatibilidad con codificaciones internacionales.
- **Puertos:** In (opcional) $\rightarrow$ RowOut .

3. DataLookupNode (Cruce de Datos / VLOOKUP)

- **Propósito:** Cruza y enriquece cada archivo contra una tabla de referencia externa (Excel, CSV, JSON) en memoria con índice hash instantáneo $O(1)$.
- **Puertos:** In $\rightarrow$ Matched , Unmatched .

4. `ExcelReportGeneratorNode` (Generador de Reportes Excel .xlsx)

- **Propósito:** Acumula los metadatos de los archivos procesados y genera una hoja de cálculo formateada al completar el flujo (`OnWorkflowCompletedAsync`).
- **Puertos:** `In` $\rightarrow$ `Out` , `Report` .

5. `CsvExportNode` (Exportador CSV / TSV)

- **Propósito:** Exporta y agrega metadatos seleccionados a un archivo plano delimitado con soporte para modo append.
- **Puertos:** `In` $\rightarrow$ `Out` .

6. `SqliteDatabaseSinkNode` (Auditoría e Inserción SQLite)

- **Propósito:** Inserta registros de auditoría y trazabilidad en una base de datos SQLite local con creación automática de esquemas e índices.
- **Puertos:** `In` $\rightarrow$ `Out` .

7. `DataFormatConverterNode` (Conversor de Formatos de Datos)

- **Propósito:** Convierte directamente archivos estructurados entre formatos `Excel (.xlsx)` $\leftrightarrow$ `CSV` $\leftrightarrow$ `JSON` .
- **Puertos:** `In` $\rightarrow$ `Out` .

Categoría 10: Scripting & Extensibility (`FileFlow.Plugin.Scripting`)

1. `CustomScriptNode` (Nodo de Scripting Dual C# & JavaScript)

- **Propósito:** Permite programar lógica de transformación a medida usando **C# (compilación JIT Roslyn en memoria)** o **JavaScript (sandbox Jint administrado)**.
- **Puertos:** Puertos dinámicos configurables por el usuario (`Inputs` y `Outputs` editables).
- **Herramienta Integrada:** Incluye **ScriptStudio**, un entorno de desarrollo integrado con editor AvalonEdit, resaltado sintáctico, consola de pruebas en vivo y biblioteca de plantillas `.ffscript` .

Categoría 11: AI & Computer Vision (`FileFlow.Plugin.AI`)

1. `LocalOcrNode` (Reconocimiento Óptico de Caracteres OCR)

- **Propósito:** Extrae texto e información estructurada desde imágenes y documentos escaneados inyectando `{Ocr:Text}` , `{Ocr:WordCount}` y `{Ocr:Language}` .
- **Puertos:** `In` $\rightarrow$ `Out` , `Error` .

2. SmartImageClassifierNode (Clasificador Visual de Fotos)

- **Propósito:** Analiza el contenido visual de fotografías e imágenes asignando categorías temáticas (Paisajes, Facturas, Retratos, Vehículos, Comida, etc.) en `{AI:Category}`, `{AI:TopLabel}` y `{AI:Confidence}`.
- **Puertos:** In $\rightarrow$ Out, Error.

3. FaceDetectorNode (Detector de Rostros Humanos)

- **Propósito:** Detecta caras humanas y bifurca el flujo según la presencia y conteo de personas (`{AI:HasFaces}` y `{AI:FaceCount}`).
- **Puertos:** In $\rightarrow$ FacesFound, NoFaces.

4. ObjectDetectorNode (Detector de Objetos YOLO)

- **Propósito:** Detecta e identifica múltiples objetos cotidianos (personas, vehículos, animales, objetos) en `{AI:DetectedObjects}` y `{AI:TopObject}`.
- **Puertos:** In $\rightarrow$ Out, Error.

5. LocalWhisperTranscriberNode (Transcriptor de Voz a Texto Whisper)

- **Propósito:** Transcribe audios y vídeos a texto y subtítulos sincronizados `.srt` de forma 100% local y privada inyectando `{Transcript}`.
- **Puertos:** In $\rightarrow$ Out, Error.

6. Modos de Ejecución Avanzados y Motor DAG

Modo Vigilante en Tiempo Real (Watchdog Trigger Mode)

- Al pulsar el botón  **Vigilante** en la barra superior, FileFlow Studio permanece escuchando en segundo plano las carpetas de origen configuradas en el flujo.
- Cada nuevo archivo copiado o descargado es detectado tras un debounce inteligente (evitando bloqueos de archivo en escritura) y procesado inmediatamente por el pipeline sin tener que reiniciar la ejecución.

Telemetría y Mapa de Cuellos de Botella (Bottleneck Heatmap)

- Medición de microsegundos en tiempo real con `Stopwatch.GetTimestamp()`.
- Badges dinámicos en la cabecera de las tarjetas de nodos que indican la latencia media ( 12 ms /  1.2 s) y resaltan en color de advertencia ( Cuello de botella) los nodos que consumen más del 35% del tiempo total del pipeline.

Modo Headless / Ejecutor CLI (`fileflow.exe --run`)

FileFlow Studio puede ejecutarse de forma desatendida desde scripts PowerShell, bash o el Programador de Tareas de Windows:

```
# Ejecución estándar con inyección de variables globales
.\FileFlow.App.exe --run "flujo.json" --input "C:\Datos" --var Environment=Production

# Sobrescritura de parámetros específicos por nodo
.\FileFlow.App.exe --run "flujo.json" --param Throttle.DelayMilliseconds=50

# Modo vigilante en consola con reporte estructurado JSON
.\FileFlow.App.exe --run "flujo.json" --watch --summary "reporte.json"

# Reanudación de flujos interrumpidos mediante puntos de control
.\FileFlow.App.exe --run "flujo.json" --resume
```

Puntos de Control y Reanudación Automática (State Checkpointing)

- Al procesar miles de archivos, FileFlow Studio guarda automáticamente puntos de control atómicos en `%LocalAppData%\FileFlowStudio/checkpoints/`.
- Si el proceso se interrumpe por corte eléctrico o reinicio del sistema, al volver a ejecutar el flujo detecta el progreso previo y **omite el reprocesamiento innecesario de los archivos ya completados exitosamente**.

7. Herramientas de Diseño y Productividad en el Lienzo

- **Notas Adhesivas (Sticky Notes):** Clic secundario en el lienzo `$\rightarrow$` `Crear Nota Adhesiva` para añadir instrucciones, comentarios o documentación técnica directamente sobre el grafo con colores personalizables y redimensionado.
- **Marcos de Agrupación Visual (Group Frames):** Selecciona múltiples nodos y presiona `Ctrl+G` para encapsularlos en un marco visual delimitado. Al mover el marco por su cabecera, todos los nodos contenidos se desplazan coordinadamente de forma interactiva.
- **Selector Desplegable de Categorías (Dropdown ComboBox):** Barra superior de herramientas compacta con búsqueda en tiempo real, filtros de favoritos (🌟), más usados (🔥) y badges numéricos reactivos por categoría.

8. Flujos de Ejemplo y Plantillas Predefinidas

Plantilla 1: Organización Fotográfica Automática

```
[FolderSourceNode]
  | (FileOut)
  ▼
[HashCalculatorNode (SHA-256)]
  | (Out)
  ▼
[AdvancedRenamerNode ({CreationDate:yyyyMMdd}_{Hash:SHA256:8}_{FileNameNoExt}.{Ext})]
  | (Out)
  ▼
[FileRelocatorNode (Destination: {SourceDir}\{Year}\{Month})]
```

Plantilla 2: Limpieza de Duplicados a la Papelera

```
[FolderSourceNode]
  | (FileOut)
  ▼
[DeduplicationFilterNode]
  ├── (Unique)    → [LogOutputNode (Archivo Único Conservado)]
  └── (Duplicate) → [SafeRecycleDeleteNode (Papelera de Windows)]
```

Plantilla 3: Cruce de Datos Excel y Envío Remoto por SFTP

```
[FolderSourceNode (PDFs)]
  | (Out)
  ▼
[DataLookupNode (clientes.xlsx por {FileNameWithoutExtension})]
  ├── (Matched)    → [SftpUploadNode (Servidor Cliente)] → [SqliteDatabaseSinkNode (Auditoría)]
  └── (Unmatched) → [FileRelocatorNode (Carpeta de Revisión Pendiente)]
```